

Φεστιβάλ

Η Νάιρα βρίσκεται σε ένα φεστιβάλ και παίζει ένα παιχνίδι που το μεγάλο έπαθλο είναι ένα ταξίδι στη Λίμνη Κολοράντα. Στο παιχνίδι χρησιμοποιούνται νομίσματα (tokens) για την αγορά κουπονιών (coupons). Όταν η Νάιρα αγοράζει ένα κουπόνι, είναι πιθανό να αποκτήσει περισσότερα νομίσματα. Ο στόχος είναι να αγοράσει όσο το δυνατόν περισσότερα κουπόνια.

Ξεκινάει το παιχνίδι με A νομίσματα. Υπάρχουν N κουπόνια, αριθμημένα από 0 έως $N - 1$. Η Νάιρα πρέπει να πληρώσει $P[i]$ νομίσματα (όπου $0 \leq i < N$) για να αγοράσει το κουπόνι i (και πρέπει να έχει τουλάχιστον $P[i]$ νομίσματα πριν από την αγορά). Μπορεί να αγοράσει κάθε κουπόνι το πολύ μία φορά.

Επιπλέον, σε κάθε κουπόνι i (όπου $0 \leq i < N$) αντιστοιχιστεί ένας **τύπος**, που συμβολίζεται με $T[i]$, ο οποίος είναι ένας ακέραιος αριθμός **μεταξύ 1 και 4, συμπεριλαμβανομένων**. Αφού η Νάιρα αγοράσει το κουπόνι i , το πλήθος των νομισμάτων που της έχουν απομείνει πολλαπλασιάζεται με $T[i]$. Τυπικά, αν έχει X νομίσματα σε κάποιο σημείο του παιχνιδιού και αγοράσει το κουπόνι i (το οποίο απαιτεί $X \geq P[i]$), τότε μετά την αγορά θα έχει $(X - P[i]) \cdot T[i]$ νομίσματα.

Η δουλειά σας είναι να προσδιορίσετε ποια κουπόνια πρέπει να αγοράσει η Νάιρα και με ποια σειρά, για να μεγιστοποιήσει τον συνολικό αριθμό **κουπονιών** που θα έχει στο τέλος. Εάν υπάρχουν περισσότερες από μία ακολουθίες αγορών που το επιτυγχάνουν αυτό, μπορείτε να αναφέρετε οποιαδήποτε από αυτές.

Λεπτομέρειες Υλοποίησης

Πρέπει να υλοποιήσετε την ακόλουθη συνάρτηση.

```
std::vector<int> max_coupons(int A, std::vector<int> P,  
std::vector<int> T)
```

- A : ο αρχικός αριθμός νομισμάτων που έχει η Νάιρα.
- P : ένας πίνακας μήκους N που καθορίζει τις τιμές των κουπονιών.
- T : ένας πίνακας μήκους N που καθορίζει τους τύπους των κουπονιών.
- Αυτή η συνάρτηση καλείται ακριβώς μία φορά για κάθε περίπτωση ελέγχου.

Η συνάρτηση επιστρέφει έναν πίνακα R , ο οποίος καθορίζει τις αγορές της Νάιρα ως εξής:

- Το μήκος του R πρέπει να είναι ο μέγιστος αριθμός κουπονιών που μπορεί να αγοράσει.
- Τα στοιχεία του πίνακα είναι οι αύξοντες αριθμοί των κουπονιών που πρέπει να αγοράσει, σε χρονολογική σειρά. Δηλαδή, αγοράζει πρώτα το κουπόνι $R[0]$, ύστερα το κουπόνι $R[1]$, και ούτω καθεξής.
- Όλα τα στοιχεία του R πρέπει να είναι μοναδικά.

Εάν η Νάιρα δεν μπορεί να αγοράσει κανένα κουπόνι, ο R πρέπει να είναι ένας κενός πίνακας.

Περιορισμοί

- $1 \leq N \leq 200\,000$
- $1 \leq A \leq 10^9$
- $1 \leq P[i] \leq 10^9$ για κάθε i τέτοιο ώστε $0 \leq i < N$.
- $1 \leq T[i] \leq 4$ για κάθε i τέτοιο ώστε $0 \leq i < N$.

Υποπροβλήματα

Υποπρόβλημα	Πόντοι	Επιπλέον περιορισμοί
1	5	$T[i] = 1$ για κάθε i τέτοιο ώστε $0 \leq i < N$.
2	7	$N \leq 3000$ και $T[i] \leq 2$ για κάθε i τέτοιο ώστε $0 \leq i < N$.
3	12	$T[i] \leq 2$ για κάθε i τέτοιο ώστε $0 \leq i < N$.
4	15	$N \leq 70$
5	27	Η Νάιρα μπορεί να αγοράσει όλα τα κουπόνια (με κάποια σειρά).
6	16	$(A - P[i]) \cdot T[i] < A$ για κάθε i τέτοιο ώστε $0 \leq i < N$.
7	18	Χωρίς επιπλέον περιορισμούς.

Παραδείγματα

Παράδειγμα 1

Έστω η ακόλουθη κλήση.

```
max_coupons(13, [4, 500, 8, 14], [1, 3, 3, 4])
```

Η Νάιρα αρχικά έχει $A = 13$ νομίσματα. Μπορεί να αγοράσει 3 κουπόνια με τη σειρά που φαίνεται παρακάτω:

Αγορασμένο κουπόνι	Τιμή κουπονιού	Τύπος κουπονιού	Αριθμός νομισμάτων μετά την αγορά
2	8	3	$(13 - 8) \cdot 3 = 15$
3	14	4	$(15 - 14) \cdot 4 = 4$
0	4	1	$(4 - 4) \cdot 1 = 0$

Σε αυτό το παράδειγμα, είναι αδυνατό για τη Νάιρα να αγοράσει περισσότερα από 3 κουπόνια και η ακολουθία αγορών που περιγράφεται παραπάνω είναι ο μόνος τρόπος με τον οποίο μπορεί να αγοράσει 3 από αυτά. Επομένως, η συνάρτηση θα πρέπει να επιστρέψει $[2, 3, 0]$.

Παράδειγμα 2

Έστω η ακόλουθη κλήση.

```
max_coupons(9, [6, 5], [2, 3])
```

Σε αυτό το παράδειγμα, η Νάιρα μπορεί να αγοράσει και τα δύο κουπόνια με οποιαδήποτε σειρά. Επομένως, η συνάρτηση θα πρέπει να επιστρέψει είτε $[0, 1]$, είτε $[1, 0]$.

Παράδειγμα 3

Έστω η ακόλουθη κλήση.

```
max_coupons(1, [2, 5, 7], [4, 3, 1])
```

Σε αυτό το παράδειγμα, η Νάιρα έχει ένα νόμισμα, το οποίο δεν είναι αρκετό για να αγοράσει κανένα κουπόνι. Επομένως, η συνάρτηση θα πρέπει να επιστρέψει $[]$ (έναν κενό πίνακα).

Υποδειγματικός Βαθμολογητής

Μορφή εισόδου:

```
N A
P[0] T[0]
P[1] T[1]
...
P[N-1] T[N-1]
```

Μορφή εξόδου:

```
S  
R[0] R[1] ... R[S-1]
```

Εδώ, το S είναι το μήκος του πίνακα R που επιστρέφεται από τη `max_coupons`.